

4. Búsqueda de hiperparámetros, validación cruzada y Control

Ejercicio 4.1.

Sea `GRID_SEARCH(X : LIST<TUPLE<ALGORITMO, HYPERS>>, D : DATA, M : METRICA) : TUPLE<ALGORITMO, HYPERS, FLOAT>` la función que ejecuta una búsqueda exhaustiva para encontrar la mejor configuración entre una lista de posibilidades y retorna el valor esperado para dicha combinación.

Detalles de los parámetros:

- X es una lista de `TUPLE<A, HS> : TUPLE<ALGORITMO, HYPERS>`, en donde A es el nombre del algoritmo y HS representa un diccionario de hiperparámetros a valores.
Por ej, un elemento de la lista puede ser `<arbol_de_decision, {altura_max : 10, medida : gini, atributos : todos}>` otro puede ser `<arbol_de_decision, {altura_max : 5, medida : entropy, atributos : todos}>`
 - D, un dataset de $n \times p$ (instancias $\times$ atributos)
 - M, una Metrica (por ejemplo, ACCURACY)
- (a) Escribir el pseudocódigo de la función. Para ello, suponga ya implementada la función `CROSS_VAL(A : ALGORITMO, HS : HYPERS, D : DATA, M : METRICA) : FLOAT` que devuelve el resultado de ejecutar validación cruzada para un algoritmo A e hiper-parámetros HS sobre la base de datos D y usando la Metrica M. Retorna el valor obtenido.
- (b) Escribir el pseudocódigo de la función `CROSS_VAL(A : ALGORITMO, HS : HYPERS, D : DATA, M : METRICA) : FLOAT`. ¿Qué tipo de validación cruzada elegiste para tu pseudocódigo?
- (c) Definir ahora la función `RANDOM_SEARCH(..., N : INT) : TUPLE<ALGORITMO, HYPERS, FLOAT>` con los mismos parámetros que `GRID_SEARCH` más N, un parámetro que indica la cantidad de intentos a probar por cada entrada de la lista. Suponer para esta función que `H : HYPERS` es un diccionario en donde los valores también pueden ser distribuciones probabilísticas a las que se las puede muestrear con la función `SAMPLE(D : DISTRIBUCIÓN) : FLOAT` (suponer que `SAMPLE(CONSTANTE) = CONSTANTE`. Ej, `<arbol_de_decision, {altura_max : Binomial(n = 20, p = 0.5), medida : entropy, atributos : todos}>`)

Ejercicio 4.2. Validación cruzada y Control. Verdadero o Falso. Justificar

- (a) Hacer validación cruzada ayuda a obtener estimaciones más realistas de la performance de un modelo sobre nuevos datos que al hacerlo sobre los mismos datos de entrenamiento.
- (b) En K-Fold Cross Validation entrenamos K modelos en donde cada modelo sólo se entrena en un subconjunto disjunto al resto de los modelos.
- (c) En K-Fold Cross Validation validamos K modelos en donde cada modelo sólo evalúa a un subconjunto disjunto al resto de los modelos
- (d) Hacer validación cruzada evita el sobreajuste (overfitting) de los modelos sobre los datos.
- (e) En K-Fold Cross Validation, conviene que K se acerque a N. De esta manera el resultado será lo más realista posible ya que se tiende a generar N modelos independientes. El problema es que hay que entrenar demasiados modelos.
- (f) Evaluar un modelo sobre el conjunto de Control resultará en un valor siempre peor o igual al conseguido en Desarrollo.
- (g) Una vez elegida la mejor configuración durante Desarrollo, es conveniente hacer K-Fold Cross Validation nuevamente, pero ahora incluyendo también el conjunto de Control.
- (h) Luego de seleccionar la mejor configuración, es ideal volver a correr el algoritmo pero esta vez utilizando todos los datos de Desarrollo para luego evaluarlo en los datos de Control.
- (i) Se espera que al evaluar un modelo sobre el Control el resultado sea peor al conseguido en Desarrollo.

Ejercicio 4.3. Preguntas para desarrollar (no hay una única respuesta correcta).

- (a) En la clase teórica vimos que al hacer Cross Validation los datos no siempre deben separarse al azar, ¿por qué?. Pensar ejemplos de al menos tres situaciones distintas en las cuales no sea conveniente.
- (b) ¿Por qué se deberían usar una sola vez los datos de Control?
- (c) ¿Qué sería conveniente hacer si luego de un muy buen resultado en Desarrollo, encuentro un pésimo resultado en Control?

Ejercicio 4.4. Consideremos K-Fold Cross Validation con las siguientes modificaciones:

- Para cada instancia $x^{(i)}$ dentro de un conjunto de datos $\mathcal{D}$ se tiene un mapeo G que le asigna un único grupo $g \in \mathcal{G}$ tal que $G(x^{(i)}) = g$ (pueden pensarlo como un $\text{DICC}(\text{INSTANCIA}, \text{GRUPO})$).
 - Al dividir $\mathcal{D}$ en los k Folds, se asegura que cada grupo g este contenido únicamente en un Fold.
- (a) Construir el algoritmo `GROUP_K_CROSS_VAL(A : ALGORITMO, HS : HYPERS, D : DATA, M : MÉTRICA, G: GRUPOS, K: INT) : FLOAT` que devuelve el resultado de evaluar el algoritmo dado con los hiperparámetros dados y algún valor de k respetando los grupos dados (suponer grupos de tamaños homogéneos).
- (b) ¿En que escenarios podría tener sentido utilizar este procedimiento? Desarrolle.
- (c) ¿Qué cambiaría en su implementación si hubiese más Folds que grupos?
- (d) ¿Qué ocurre si un único grupo concentra el 60 % de todas las instancias del dataset?

Ejercicio 4.5. Luego de correr un Grid Search (búsqueda de hiperparámetros) en donde evaluamos miles de configuraciones (utilizando Cross Validation, con buenas particiones), encontramos una que funciona mejor que el resto con un 93 % de Accuracy en los Folds para validación (el resto no superaban el 92.9 %). Este resultado, comparado contra datos nuevos de la realidad (o en datos de Control), será generalmente (puede haber más de una correcta):

- (a) demasiado **optimista** (porque el modelo puede haber sobreajustado a los datos de entrenamiento de cada Fold).
- (b) demasiado **optimista** (al probar tantas combinaciones de modelos, es posible que la configuración seleccionada haya tenido suerte en esos datos particulares o en la aleatoriedad misma de la configuración ejecutada – ej, random seeds).
- (c) **acertado** ya que si hicimos bien los splits en Cross Validation, podemos garantizar que no hay información que se esté filtrando entre las distintas particiones y por lo tanto deberíamos obtener el mismo valor en datos nuevos.
- (d) demasiado **optimista** (porque habremos usado los datos de validación para tomar decisiones respecto al sistema).
- (e) **optimista** y es por eso que sería ideal evaluar en datos frescos (Control) antes de reportar el resultado.
- (f) **pesimista** porque los datos elegidos para Desarrollo se buscan que sean más complejos que los de la realidad para prevenir sorpresas.

Ejercicio 4.6. La técnica de SOBREMUESTREO (oversampling) consiste en crear copias de instancias al azar (a veces agregando mínimas modificaciones en alguno de sus atributos) y asignarle la etiqueta original.

Este proceso se utiliza mucho cuando las clases están muy desbalanceadas y no es suficiente la cantidad de instancias de alguna clase para entrenar un clasificador. En estos casos es normal, por ejemplo, sobremuestrear hasta lograr la misma cantidad de instancias para cada clase.

Por ejemplo, dado el problema de clasificar entre GATOS, PERROS y CONEJOS, y dado que la cantidad de instancias es 100, 2000 y 20 respectivamente, generamos un nuevo dataset de 2000, 2000, y 2000 instancias, en donde las instancias agregadas son copias de instancias de la clase correspondiente elegidas al azar y a las que se le agrega un poco de ruido a sus columnas numéricas.

Describir los pros y contras de aplicar la técnica en los siguientes pasos del proceso. Justificar en términos de posibilidades de sub o sobreestimar la performance de nuestros modelos y concluir cuál es la opción más segura:

- (I) Antes de partir en Desarrollo - Control
- (II) Antes de partir en Folds en Desarrollo.
- (III) Luego de seleccionar los Folds que se utilizarán para entrenar el modelo dentro de las iteraciones de K-Fold Cross Validation. Aplicarlo sólo a los datos de entrenamiento.
- (IV) Luego de seleccionar los Folds que se utilizarán para entrenar el modelo dentro de las iteraciones de K-Fold Cross Validation. Aplicarlo tanto a los datos de entrenamiento como a los de validación.

Ejercicio 4.7. Ordenar los siguientes pasos del Desarrollo y Control de un modelo de manera apropiada (indentar si hay loops y especificar sobre qué datos se haría)

- (a) Mandar reporte final a toda la empresa y submittear paper.
- (b) Partir los datos en Desarrollo - Control
- (c) Entrenar un modelo usando una configuración. Exportar a disco el archivo con los parámetros aprendidos del modelo final (modelo.zip).
- (d) Partir en 10 Folds.
- (e) Correr un proceso que lista los atributos según qué tanto se correlacionan con las etiquetas y conservar sólo el top-10 para entrenar el modelo.
- (f) Inspeccionar las instancias donde el modelo cometió los errores más graves para diseñar nuevas transformaciones de atributos.
- (g) Definir la Metrica a utilizar (ej: Accuracy pesada por clase)
- (h) Mirar / escuchar / leer instancias del dataset para pensar buenos atributos.
- (i) No me dio tan bien como esperaba. Empiezo de nuevo repensando algunas cosas.
- (j) Armar una grilla de configuraciones a probar
- (k) Después de correr lo anterior me di cuenta que podría haber probado tal modelo / tal hiperparámetro. Agregar a las configuraciones y repetir.
- (l) Graficar la distribución de las etiquetas.
- (m) Calcular el promedio de la columna 'edad' para reemplazar los valores nulos en los registros que no la tengan.
- (n) Calcular la performance de un modelo trivial (baseline, e.g., predecir siempre la clase mayoritaria para clasificación o la media para regresión)

Ejercicio 4.8. Consideremos la evaluación de un modelo mediante K -Fold Cross Validation sobre un conjunto de datos $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, donde para cada instancia $x^{(i)}$ se obtiene su predicción *out-of-fold* $\hat{y}^{(i)}$. Se plantean dos estrategias para reportar una métrica de performance $\mathcal{M}$:

- **Cálculo global:** computar $\mathcal{M}$ evaluando directamente todas las predicciones sobre el conjunto total $\mathcal{D}$.
- **Promedio por folds:** calcular $\mathcal{M}_k$ en cada fold de validación $\mathcal{D}_k$ ($k = 1, \dots, K$) y luego promediar dichos valores: $\frac{1}{K} \sum_{k=1}^K \mathcal{M}_k$.

Sean $\mathcal{D}_1, \dots, \mathcal{D}_K$ una partición de $\mathcal{D}$ tal que $|\mathcal{D}_k| = n_k$ y $\sum_{k=1}^K n_k = N$.

- (a) Demostrar que si todos los folds tienen el mismo tamaño ($n_k = \frac{N}{K}$ para todo k), el cálculo global coincide exactamente con el promedio simple de los folds para las siguientes dos métricas aditivas:
 - **Accuracy** (clasificación): $\text{Acc}(\mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \mathbb{I}(\hat{y}^{(i)} = y^{(i)})$.
 - **MSE** (regresión): $\text{MSE}(\mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} (y^{(i)} - \hat{y}^{(i)})^2$.
- (b) ¿Qué ocurre si los folds tienen diferente tamaño ($n_i \neq n_j$)? Demostrar qué ponderación sobre los resultados locales permite recuperar la métrica global.
- (c) Consideremos ahora la métrica **RMSE** (raíz del error cuadrático medio), definida como $\text{RMSE}(\mathcal{S}) = \sqrt{\text{MSE}(\mathcal{S})}$. Construir un contraejemplo simple con $K = 2$ folds de igual tamaño ($n_1 = n_2$) donde el promedio simple de los RMSE locales difiera del RMSE global. ¿Cuál de los dos valores es mayor o igual y por qué?

Ejercicio 4.9. Completar el notebook `notebook_seleccion_modelos.ipynb`